tf.one_hot Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/one_hot

> *Returns a one-hot tensor.*

Function Signatures

```
tf.one_hot( indices, depth, on_value=None, off_value=None,
axis=None, dtype=None, name=None )
```

Code Examples

```
tf.one_hot(
indices,
depth,
on_value=None,
off_value=None,
axis=None,
dtype=None,
name=None
)
```

```
tf.one_hot(
indices,
depth,
on_value=None,
off_value=None,
axis=None,
dtype=None,
name=None
)
```

```
features x depth if axis == -1
depth x features if axis == 0
```


```
features x depth if axis == -1
depth x features if axis == 0
```


```
batch x features x depth if axis == -1
batch x depth x features if axis == 1
depth x batch x features if axis == 0
```


```
batch x features x depth if axis == -1
batch x depth x features if axis == 1
depth x batch x features if axis == 0
```

```
indices = [0, 1, 2]
depth = 3
tf.one_hot(indices, depth)  # output: [3 x 3]
# [[1., 0., 0.],
#  [0., 1., 0.],
#  [0., 0., 1.]]
indices = [0, 2, -1, 1]
depth = 3
tf.one_hot(indices, depth,
on_value=5.0, off_value=0.0,
axis=-1)  # output: [4 x 3]
# [[5.0, 0.0, 0.0],  # one_hot(0)
#  [0.0, 0.0, 5.0],  # one_hot(2)
#  [0.0, 0.0, 0.0],  # one_hot(-1)
#  [0.0, 5.0, 0.0]]  # one_hot(1)
indices = [[0, 2], [1, -1]]
depth = 3
tf.one_hot(indices, depth,
on_value=1.0, off_value=0.0,
axis=-1)  # output: [2 x 2 x 3]
# [[[1.0, 0.0, 0.0],   # one_hot(0)
#   [0.0, 0.0, 1.0]],  # one_hot(2)
#  [[0.0, 1.0, 0.0],   # one_hot(1)
#   [0.0, 0.0, 0.0]]]  # one_hot(-1)
indices = tf.ragged.constant([[0, 1], [2]])
depth = 3
tf.one_hot(indices, depth)  # output: [2 x None x 3]
# [[[1., 0., 0.],
#   [0., 1., 0.]],
#  [[0., 0., 1.]]]
```

```
indices = [0, 1, 2]
depth = 3
tf.one_hot(indices, depth)  # output: [3 x 3]
# [[1., 0., 0.],
#  [0., 1., 0.],
#  [0., 0., 1.]]
indices = [0, 2, -1, 1]
depth = 3
tf.one_hot(indices, depth,
on_value=5.0, off_value=0.0,
axis=-1)  # output: [4 x 3]
# [[5.0, 0.0, 0.0],  # one_hot(0)
#  [0.0, 0.0, 5.0],  # one_hot(2)
#  [0.0, 0.0, 0.0],  # one_hot(-1)
#  [0.0, 5.0, 0.0]]  # one_hot(1)
indices = [[0, 2], [1, -1]]
depth = 3
tf.one_hot(indices, depth,
on_value=1.0, off_value=0.0,
axis=-1)  # output: [2 x 2 x 3]
# [[[1.0, 0.0, 0.0],   # one_hot(0)
#   [0.0, 0.0, 1.0]],  # one_hot(2)
#  [[0.0, 1.0, 0.0],   # one_hot(1)
#   [0.0, 0.0, 0.0]]]  # one_hot(-1)
indices = tf.ragged.constant([[0, 1], [2]])
depth = 3
tf.one_hot(indices, depth)  # output: [2 x None x 3]
# [[[1., 0., 0.],
#   [0., 1., 0.]],
#  [[0., 0., 1.]]]
```

Documentation

Returns a one-hot tensor.

View aliases

Compat aliases for migration

See
Migration guide for
more details.

tf.compat.v1.one_hot

Used in the notebooks

• Import a JAX model using JAX2TF

- Quickstart for the TensorFlow Core APIs
- Training & evaluation with the built-in methods
- Adversarial example using FGSM
- Custom Federated Algorithms, Part 2: Implementing Federated Averaging
- Federated Learning for Image Classification
- Client-efficient large-model federated learning via `federated_select` and sparse aggregation
- Training with Orbit

See also tf.fill, tf.eye.

The locations represented by indices in indices take value on_value,
while all other locations take value off_value.

on_value and off_value must have matching data types. If dtype is also
provided, they must be the same data type as specified by dtype.

If on_value is not provided, it will default to the value 1 with type
dtype

If off_value is not provided, it will default to the value 0 with type dtype

If the input indices is rank N, the output will have rank N+1. The new axis is created at dimension axis (default: the new axis is appended at the end).

If indices is a scalar the output shape will be a vector of length depth

If indices is a vector of length features, the output shape will be:

If indices is a matrix (batch) with shape [batch, features], the output shape will be:

If indices is a RaggedTensor, the 'axis' argument must be positive and refer to a non-ragged axis. The output will be equivalent to applying 'one_hot' on the values of the RaggedTensor, and creating a new RaggedTensor from the result.

If dtype is not provided, it will attempt to assume the data type of on_value or off_value, if one or both are passed in. If none of on_value, off_value, or dtype are provided, dtype will default to the value tf.float32.

For example:

Returns

Raises